

PDR

TV5 – PAYMENT SERVICE

Node.js · MongoDB · Cổng 5005

Thông tin	Giá trị
Dự án	Hệ thống đặt vé xem phim theo kiến trúc Microservices
Vai trò	TV5 – Xử lý thanh toán giả lập và lưu lịch sử giao dịch
Giữa kỳ	Node.js 18+ · module http · driver mongodb
Cuối kỳ	Express · Mongoose · jsonwebtoken
CSDL	MongoDB payment_db · collection transactions
Tài liệu tham chiếu	Kế hoạch triển khai SOA Đặt vé phim v2.1, cập nhật 02/10/2026

Họ tên / MSSV: _____

Phiên bản PDR: 1.0 | Ngày lập: 04/10/2026

Mục tiêu tài liệu

Tài liệu này chuyển yêu cầu phiếu riêng TV5 thành danh sách công việc có thể thực hiện, tiêu chí nghiệm thu, cấu trúc bàn giao và kịch bản kiểm thử. Phạm vi giữa kỳ được tách khỏi backlog cuối kỳ để bảo đảm đúng lộ trình “code thuần → framework”.

1. Tổng quan và phạm vi

TV5 sở hữu trọn vẹn Payment Service: mô hình dữ liệu, API, sơ đồ phân tích, trang admin và báo cáo. Service nhận yêu cầu thanh toán từ Booking Service, mô phỏng kết quả, lưu giao dịch vào MongoDB và cho phép Admin xem lịch sử.

Phạm vi giữa kỳ: Node.js 18+, module http và driver mongodb; chưa dùng Express/Mongoose, chưa tích hợp cổng thanh toán thật và chưa bắt buộc JWT thật. Phạm vi cuối kỳ: chuyển sang Express + Mongoose, bổ sung middleware verify JWT và giới hạn GET cho ADMIN.

Hạng mục	Quy định phải tuân thủ
Cổng / bind	5005 / 0.0.0.0 để máy khác trong nhóm có thể gọi
CSDL	MongoDB payment_db; TV5 là service duy nhất sở hữu collection transactions
FK liên service	bookingId là FK logic tới Booking.bookings.id; không tạo FK vật lý
Định dạng	JSON; Content-Type: application/json; charset=utf-8
CORS	Xử lý GET, POST, OPTIONS; OPTIONS trả 204
HTTP chính	201 tạo giao dịch; 200 lấy lịch sử; 400 dữ liệu/JSON sai; 405 sai method; 500 lỗi server

2. Danh sách đầu ra P1-P8

Mã	Đầu ra	Công việc cần hoàn thành	Tiêu chí nghiệm thu
P1	CSDL + seed	Tạo payment_db.transactions; viết seed.js chạy bằng mongosh; nạp 2 giao dịch Phụ lục A.5.	Có đúng các dòng (1, 180000, MOMO, PAID) và (2, 85000, VNPay, FAILED); bookingId là Number; paidAt là Date.
P2	ERD	Vẽ collection như một thực thể; ghi rõ kiểu dữ liệu; thể hiện FK logic.	Đủ 6 trường: _id:ObjectId, bookingId:Number, amount:Number, method:String, status:String, paidAt:Date.
P3	Use Case	Actor Booking Service và Admin; xử lý, lưu, xem lịch sử.	Có sơ đồ và đặc tả ngắn cho 3 use case.
P4	DFD mức 0/1	Mức 0: Booking/Admin ↔ Payment. Mức 1: tiếp nhận, xử lý/ghi, truy vấn.	Có kho D1 – transactions và luồng vào/ra.
P5	DFD mức 2	Phân rã 2.0: 2.1 kiểm tra; 2.2 giả lập; 2.3 ghi; 2.4 phản hồi.	Thể hiện cả nhánh PAID và FAILED.
P6	API	POST /api/payment và GET /api/payment bằng http thuần; body data/end + JSON.parse try/catch.	API chạy, đúng mã HTTP, lưu cả giao dịch FAILED, xử lý OPTIONS.
P7	Trang admin	Tạo admin_payment.html; bảng 5 trường và thông báo lỗi thân thiện.	Gọi GET và hiển thị dữ liệu từ service.

Mã	Đầu ra	Công việc cần hoàn thành	Tiêu chí nghiệm thu
P8	Báo cáo	Mô tả kiến trúc, dữ liệu, API, sơ đồ, ảnh Compass, ảnh curl và hướng dẫn chạy.	Báo cáo có minh chứng, checklist và nêu giới hạn giữa kỳ.

3. Thiết kế dữ liệu và seed

3.1. Collection transactions

Trường	Kiểu	Bắt buộc	Quy tắc / ý nghĩa
_id	ObjectId	Tự sinh	Định danh MongoDB.
bookingId	Number	Có	FK logic → Booking.bookings.id; tuyệt đối không lưu chuỗi.
amount	Number	Có	Số tiền VND; > 0 là nhánh PAID.
method	String	Có	Chỉ MOMO, VNPAY hoặc CASH.
status	String	Có	Chỉ PAID hoặc FAILED.
paidAt	Date	Có	Thời điểm xử lý, dùng Date/ISODate.

Ghi chú nhất quán dữ liệu: Payment Service không tạo khóa ngoại thật sang booking_db. Booking Service chịu trách nhiệm truyền bookingId hợp lệ; TV5 lưu dạng Number để khớp với bookings.id.

3.2. Dữ liệu seed bắt buộc theo Phụ lục A.5

```
use payment_db;
db.transactions.deleteMany({});
db.transactions.insertMany([
  { bookingId: 1, amount: 180000, method: "MOMO", status: "PAID", paidAt: ISODate("2026-10-01T10:00:00Z") },
  { bookingId: 2, amount: 85000, method: "VNPAY", status: "FAILED", paidAt: ISODate("2026-10-01T10:05:00Z") }
]);
```

- Lưu file tại seed/payment_seed.js hoặc payment-service/seed.js; chạy bằng mongosh, không phải node seed.js.
- Sau khi seed, mở MongoDB Compass, chọn payment_db → transactions và chụp ảnh đủ các trường để đưa vào P8.
- Không đổi bookingId, amount, method, status của hai giao dịch mẫu; paidAt có thể dùng thời điểm cố định để ảnh minh chứng tải lập được.

4. Đặc tả Use Case và DFD

Use Case	Actor	Tiền điều kiện	Kết quả
UC1 – Xử lý thanh toán	Booking Service	Có bookingId, amount, method.	Tạo giao dịch PAID nếu hợp lệ; FAILED nếu amount ≤ 0 hoặc method sai.

Use Case	Actor	Tiền điều kiện	Kết quả
UC2 – Lưu giao dịch	Payment Service	Đã xác định status và paidAt.	Insert vào D1 – transactions; trả giao dịch đã lưu.
UC3 – Xem lịch sử	Admin	Payment Service đang chạy.	GET trả mảng giao dịch; admin hiển thị trong bảng.

Yêu cầu sơ đồ: DFD0 thể hiện Booking Service và Admin trao đổi với Payment Service. DFD1 gồm 1.0 Tiếp nhận yêu cầu thanh toán, 2.0 Xử lý và ghi giao dịch, 3.0 Truy vấn lịch sử và kho D1 transactions. DFD2 phân rã 2.0 thành 2.1 Kiểm tra số tiền/phương thức, 2.2 Giả lập cổng thanh toán, 2.3 Ghi giao dịch, 2.4 Tạo phản hồi.

5. Đặc tả API giữa kỳ

Method & URL	Request	Xử lý	Response kỳ vọng
POST /api/payment	{ bookingId, amount, method }	Validate; hợp lệ → PAID; sai → FAILED; luôn insert.	201 + giao dịch đã lưu.
GET /api/payment	Không có body	Đọc transactions; nên sắp xếp paidAt giảm dần.	200 + mảng giao dịch.
OPTIONS /api/payment	Preflight	Trả CORS headers.	204, không body.

5.1. Quy tắc xử lý body và lỗi

- Dùng req.on("data", ...) để nối chuỗi body và req.on("end", ...) để JSON.parse trong try/catch.
- JSON sai cú pháp phải trả 400 và không được làm sập tiến trình Node.js.
- method hợp lệ là MOMO, VNPAY, CASH; nên chuẩn hóa bằng String(method).toUpperCase() nhưng phải ghi rõ trong báo cáo.
- amount phải là Number và lớn hơn 0. amount ≤ 0 hoặc method không hợp lệ vẫn được lưu với status FAILED.
- Cấu trúc trả về nên thống nhất với nhóm: { "status": "success", "data": ... } hoặc thông báo lỗi { "status": "error", "message": "..." }.

5.2. Ví dụ request và curl

```
curl -X POST http://localhost:5005/api/payment \
  -H "Content-Type: application/json" \
  -d '{"bookingId":3,"amount":180000,"method":"MOMO"}'
```

```
curl http://localhost:5005/api/payment
```

```
# Nhánh FAILED: amount không hợp lệ
curl -X POST http://localhost:5005/api/payment \
  -H "Content-Type: application/json" \
  -d '{"bookingId":3,"amount":0,"method":"MOMO"}'
```

```
# Nhánh FAILED: method không hợp lệ
curl -X POST http://localhost:5005/api/payment \
  -H "Content-Type: application/json" \
  -d '{"bookingId":3,"amount":180000,"method":"BANK"}'
```

6. Kế hoạch cấu trúc mã nguồn và admin

```
payment-service/  
├── payment_service.js      # server module http, router, API handlers  
├── seed.js                 # seed chạy bằng mongosh (hoặc file seed/payment_seed.js)  
├── package.json  
├── config.example          # MONGO_URI, PORT, JWT_SECRET cho cuối kỳ  
├── .gitignore  
└── admin_payment.html      # bảng lịch sử giao dịch
```

Thành phần	Việc cần làm	Hoàn thành khi
Server	Listen 0.0.0.0:5005; route theo method + pathname.	node payment_service.js khởi động không lỗi.
MongoDB	Connect payment_db; insert/find transactions.	Compass thấy dữ liệu và API đọc được.
Validation	Kiểm tra bookingId, amount, method; gán PAID/FAILED.	Có test hợp lệ, amount ≤ 0 và method sai.
CORS	Headers cho Origin/Methods/Headers; OPTIONS 204.	Admin gọi API không bị chặn preflight.
Admin	Fetch GET; render bảng bookingId, tiền VND, phương thức, trạng thái, thời gian.	Có empty state và thông báo service chưa chạy.

7. Kiểm thử và minh chứng

ID	Kịch bản	Input / thao tác	Kỳ vọng
T01	PAID – MOMO	bookingId=3, amount=180000, method=MOMO	201; status PAID; lưu Number/Date.
T02	PAID – VNPAY	bookingId=4, amount=85000, method=VNPAY	201; status PAID.
T03	FAILED – amount 0	amount=0, method=MOMO	FAILED; vẫn có bản ghi trong DB.
T04	FAILED – amount âm	amount=-1, method=CASH	FAILED; vẫn có bản ghi.
T05	FAILED – method sai	method=BANK	FAILED; vẫn có bản ghi.
T06	JSON hỏng	Body {"bookingId":	400; server vẫn chạy.
T07	GET history	curl GET /api/payment	200; body là mảng giao dịch.
T08	CORS preflight	curl -X OPTIONS /api/payment	204 và đủ headers CORS.

Ảnh cần đưa vào báo cáo: (1) terminal chạy server, (2) curl POST nhánh PAID, (3) curl POST nhánh FAILED, (4) curl GET, (5) MongoDB Compass hiển thị hai seed và các giao dịch test, (6) trang admin.

8. Phân công, mốc hoàn thành và bàn giao

Mốc	Việc TV5	Sản phẩm bàn giao
M1 – Chuẩn bị	Tạo DB, collection, seed; hoàn thiện ERD nháp.	seed.js, ảnh Compass, ERD nguồn + PNG.

Mốc	Việc TV5	Sản phẩm bàn giao
M2 – API core	Implement http, body parser, validation, insert/find, CORS.	payment_service.js chạy cổng 5005.
M3 – Admin + test	Làm admin_payment.html; chạy T01–T08; chụp minh chứng.	admin_payment.html, log/curl, ảnh kiểm thử.
M4 – Hồ sơ	Viết Use Case, DFD0/1/2, báo cáo P8; rà checklist.	docs/tv5-payment/ đầy đủ P1–P8.
M5 – Ghép nhóm	Bàn giao hợp đồng endpoint cho TV4/TV6; hỗ trợ tab admin.	URL, request/response, CORS, dữ liệu seed và lưu ý tích hợp.

Đề xuất cấu trúc repo theo kế hoạch chung: docs/tv5-payment/, payment-service/, seed/payment_seed.js; tên sơ đồ: ERD_payment.png, UC_payment.png, DFD0_payment.png, DFD1_payment.png, DFD2_payment.png và lưu kèm file nguồn draw.io.

9. Backlog cuối kỳ

- Chuyển HTTP router sang Express và MongoDB driver sang Mongoose model/schema.
- Thêm middleware verify JWT bằng jsonwebtoken; GET /api/payment chỉ cho role ADMIN.
- Giữ nguyên hợp đồng POST /api/payment để Booking Service có thể gọi; Booking chuyển tiếp Authorization token.
- Bổ sung timeout/error handling khi Booking gọi Payment; không làm mất trạng thái đơn khi Payment hoặc Notification lỗi.
- Thêm test tự động, logging, biến môi trường và tài liệu triển khai.

10. Checklist tự kiểm tra trước khi nộp

- ☐ P1: payment_db và transactions tạo đúng; seed A.5 chạy bằng mongosh.
- ☐ P2: ERD ghi rõ ObjectId, Number, String, Date và FK logic bookingId.
- ☐ P3: Use Case có Booking Service, Admin và 3 use case chính.
- ☐ P4: DFD mức 0 và mức 1 có D1 – transactions.
- ☐ P5: DFD mức 2 có 2.1–2.4 và hai nhánh PAID/FAILED.
- ☐ P6: POST/GET chạy trên 0.0.0.0:5005; xử lý body data/end, JSON.parse try/catch, OPTIONS.
- ☐ P7: admin_payment.html hiển thị đủ bookingId, amount, method, status, paidAt.
- ☐ P8: Báo cáo có hướng dẫn chạy, ảnh Compass, ảnh curl và ghi rõ giới hạn giữa kỳ.
- ☐ bookingId được lưu dạng Number; không để lộ mật khẩu hoặc khóa bí mật trong repo.

11. Kết luận

Hoàn thành PDR này đồng nghĩa TV5 có một Payment Service độc lập, có dữ liệu mẫu tái lập được, API đúng hợp đồng nhóm, sơ đồ phân tích đủ P3–P5, trang admin và bộ minh chứng P6–P8. Điểm cần ưu tiên kiểm tra là nhánh FAILED vẫn phải được lưu, bookingId phải là Number và server http thuần phải tự xử lý body/JSON an toàn.